UNIVERSIDADE ABERTA

Formas Normais: da Lógica Matemática à Computação

Olivia Pachele Mattiazzo

Mestrado em Estatística, Matemática e Computação

Orientadores: Prof.ª Dr.ª Gilda Ferreira
e Prof.ª Dr.ª Yves Robert

Novembro 2026

UNIVERSIDADE ABERTA

Formas Normais: da Lógica Matemática à Computação

Olivia Pachele Mattiazzo

Mestrado em Estatística, Matemática e Computação

Novembro 2026

Creative Commons License

Agradecimentos

Resumo

Palavras-chave:

Abstract

Keywords:

Resumo Alargado em Português

Índice

Introdução **27**

1 Contexto Histórico e Fundamentos Teóricos **29**
1.1 Origens das formas normais na lógica 29
1.2 Desenvolvimento e importância na computação 30
1.3 Definições e propriedades: negativa, conjuntiva, disjuntiva e prenexa 31
1.4 Completude funcional: bases de conectivos 33
1.5 Limitações teóricas 35

2 Algoritmos de Transformação **37**
2.1 Revisão dos principais métodos de conversão para formas normais 37
2.2 Correção, equivalência lógica e complexidade computacional 37
2.3 Considerações sobre implementação computacional 37

3 Projeto e Implementação da Ferramenta **38**
3.1 Requisitos funcionais e arquitetura da solução 38
3.2 Módulos de transformação algorítmica 38
3.3 Interface pedagógica 38
3.4 Validação: testes automatizados 38

4 Conclusão **39**
4.1 Síntese dos resultados obtidos 39
4.2 Contribuições para o ensino de lógica 39
4.3 Limitações do trabalho realizado 39
4.4 Perspetivas de evolução e trabalhos futuros 39

References **40**

A Scripts de Transformação **43**
A.1 Scripts de Transformação 43

Lista de Tabelas

Lista de Figuras

Lista de Abreviações e Acrónimos

CNF Conjunctive Normal Form (Forma Normal Conjuntiva)

coNP Complemento de NP (classe de complexidade)

DNF Disjunctive Normal Form (Forma Normal Disjuntiva)

DPLL Davis–Putnam–Logemann–Loveland (algoritmo)

NAND Negação Conjunta (*Not AND*; traço de Sheffer)

NNF Negation Normal Form (Forma Normal Negativa)

NOR Negação Disjunta (*Not OR*; seta de Peirce)

NP Nondeterministic Polynomial time (classe de complexidade)

P Polynomial time (classe de complexidade)

PNF Prenex Normal Form (Forma Normal Prenexa)

SAT Boolean Satisfiability Problem (Problema de Satisfazibilidade Booleana)

Introdução

Apesar de não vermos explicitamente os conceitos matemáticos mais complexos em ação quando usamos uma interface de um sistema operacional ou quando jogamos videogame, por exemplo, eles estão lá por debaixo dos panos. Não é possível imaginar aprimoramento e refinamento de algoritmos para uma melhor perfomance sem levar em conta os pilares matemáticos que estão por trás do digital. Tudo o que temos em termos de tecnologia, desde as mais básicas como linguagem de máquina, até as mais avançadas, como as polêmicas Inteligências Artificiais generativas, tem seus pilares sustentados em conceitos matemáticos desenvolvidos a base de muito papel e caneta. Podem parecer conceitos antigos e/ou ultrapassados mas, sem eles, não teríamos estrutura para o passo a frente em termos de evolução tecnológica.

Walicki, em *Introduction to Mathematical Logic* (2011) [12], considera que o conceito de lógica foi inventado por Parmenides, no Egito, pois foi o primeiro filósofo que não somente propôs uma visão da realidade mas também a apoiou em uma extensa argumentação. Passando por muitos filósofos e matemáticos da antiguidade, chegamos eventualmente no período entre a segunda metade do século XIX e meados do século XX, onde os sistemas e cálculo de Leibniz marcam a aparição de uma linguagem formalizada e simbólica mais alinhada com a manipulação matemática. A partir disso, passamos por Geoge Boole, Gottlob Frege, Russell e Whitehead, que serão mais aprofundados conforme esse trabalho avança, mas que desenvolveram, formalizaram e consolidaram a manipulação formal de fórmulas. No cenário desenvolvido por estes mestres, as formas normais passaram a desempenhar um papel central na análise da dedução.

Um dos desafios de hoje, no entanto, se dá no ensino da lógica matemática e computacional aos novos alunos dos cursos de exatas. Existem ferramentas que auxiliam no entendimento, como ambientes de programação visual e ferramentas para aprendizado de lógica de programação, como, por exemplo, o *Scratch, Inventor, AgentSheets, RoboMind*, etc. [4] [11] Porém, existem dois principais obstáculos no uso dessas ferramentas: primeiro que tem uma abordagem mais infantil do ensino, e segundo, estão mais focados na lógica computacional e de programação do que no ensino de lógica matemática em si. Há uma lacuna que este trabalho visa preencher em termos de ferramenta pedagógica para auxiliar no aprendizado de formas normais em um contexto mais acadêmico.

O objetivo geral desta dissertação é examinar as formas normais no âmbito da lógica matemática, situando seu surgimento no contexto histórico da formalização da matemática entre o final do século XIX e o início do século XX. Busca-se compreender como a sistematização das linguagens formais, especialmente a partir dos trabalhos de Boole, Frege, Russell e seus sucessores, contribuiu para a consolidação de procedimentos de transformação de fórmulas, culminando na definição e no uso das formas normais. Além da análise histórica e conceitual, pretende-se investigar a relevância dessas estruturas na computação contemporânea. Por fim, o trabalho propõe discutir as principais dificuldades enfrentadas por estudantes no processo de aprendizagem da lógica matemática e computacional.

Como desdobramento dos objetivos gerais, esta dissertação tem como objetivo específico conceber e implementar uma ferramenta pedagógica destinada a apoiar a compreensão das transformações envolvidas na obtenção das formas normais conjuntiva e disjuntiva. Para além da construção do sistema, o trabalho registra e analisa as decisões técnicas que fundamentaram sua arquitetura e funcionamento, bem como os procedimentos de teste adotados, buscando assegurar consistência, correção lógica e adequação didática. Dessa forma, a ferramenta não se configura apenas como um produto computacional, mas também como um artefato de investigação no âmbito do ensino de lógica.

Capítulo 1

Contexto Histórico e Fundamentos Teóricos

1.1 Origens das formas normais na lógica

Nos finais do século XIX e início do século XX, a matemática começou a passar por uma transformação significativa, com ênfase crescente na rigorosidade e no método axiomático, estabelecendo as bases para a lógica matemática moderna. A chamada Lógica Simbólica Moderna foi desenvolvida, em parte, como uma maneira de prover uma estrutura formal para a manipulação de ideias e argumentos matemáticos. Depois de alguns avanços, também ganhou o objetivo de reduzir expressões lógicas a padrões mais simples que preservassem a equivalência lógica das demonstrações formais. A lógica também influenciou a ciência da computação, especialmente em áreas como raciocínio automatizado, verificação de programas e sistemas, e sistemas de tipos para linguagens de programação. Exemplos incluem o uso de álgebra booleana em circuitos de comutação, algoritmos para prova automática de teoremas e o desenvolvimento de linguagens de programação tipadas. [13]

Tradicionalmente, a história da lógica moderna é narrada como a história da lógica matemática ou simbólica, que foi criada por matemáticos. George Boole é apontado como um dos principais pioneiros dessa área, com sua obra "The Mathematical Analysis of Logic"(1847), que marcou o início do desenvolvimento contínuo da lógica matemática. Boole introduziu a ideia de representar proposições lógicas por meio de símbolos e operações matemáticas, criando o que ele chamou de "Cálculo da Lógica". Ele buscava estabelecer uma base psicológica para a lógica, mediada pela linguagem, e propôs que a lógica deveria ser associada à matemática, e não à metafísica. Sua abordagem inovadora permitiu que proposições lógicas fossem tratadas como teoremas matemáticos, utilizando leis de combinação semelhantes às da álgebra simbólica.

A lógica matemática de Boole foi inicialmente ignorada por muitos matemáticos e

filósofos, devido à falta de tradição em lógica formal na Grã-Bretanha e à visão predominante na Alemanha de que a lógica formal já era um campo completo desde Aristóteles. No entanto, após sua morte em 1864, o trabalho de Boole começou a ganhar reconhecimento, especialmente por meio de William Stanley Jevons, que simplificou e popularizou o cálculo lógico de Boole, e Alexander Bain, que introduziu as contribuições de Boole em sua obra sobre lógica. A lógica matemática de Boole foi fundamental para o desenvolvimento da lógica simbólica como uma subdisciplina da matemática no início do século XX. Sua abordagem abriu caminho para aplicações em diversas áreas, como fundamentos da matemática, física, teoria das probabilidades e filosofia da ciência. [5]

1.2 Desenvolvimento e importância na computação

Se a seção anterior situou o surgimento das formas normais no interior do próprio desenvolvimento da lógica simbólica, cabe agora examinar como esse aparato conceitual migrou da matemática pura para o domínio da computação, tornando-se um dos seus pilares operacionais. O ponto de inflexão costuma ser atribuído a Claude Shannon que, em sua dissertação de mestrado *A Symbolic Analysis of Relay and Switching Circuits* (1938), demonstrou que a álgebra booleana desenvolvida por Boole podia servir de base para a análise e a síntese de circuitos de relés e comutação elétrica [8]. Ao representar estados de circuito por meio de variáveis binárias e operações lógicas, Shannon evidenciou que problemas de engenharia elétrica podiam ser tratados como problemas puramente algébricos, e vice-versa: circuitos podiam ser projetados a partir da manipulação simbólica de expressões booleanas. Essa correspondência tornou as formas normais, sobretudo a conjuntiva e a disjuntiva, instrumentos naturais de trabalho, uma vez que qualquer função booleana pode ser expressa de modo sistemático como uma disjunção de conjunções (soma de produtos) ou como uma conjunção de disjunções (produto de somas), fornecendo assim um procedimento uniforme para simplificação e implementação de circuitos lógicos.

A relevância computacional das formas normais se aprofunda ainda mais com o advento do raciocínio automatizado. Em 1960, Martin Davis e Hilary Putnam propuseram um procedimento para testar a satisfazibilidade de fórmulas da lógica de primeira ordem que tomava como entrada expressões já convertidas para a forma normal conjuntiva, tratando cada fórmula como uma conjunção de cláusulas disjuntivas de literais [3]. O procedimento de Davis e Putnam, posteriormente refinado por Davis, Logemann e Loveland no algoritmo hoje conhecido como DPLL, consolidou a forma normal conjuntiva como o formato padrão de entrada para os problemas de satisfazibilidade booleana (SAT), estabelecendo um paradigma que permanece no núcleo dos solucionadores de SAT contemporâneos, empregados em verificação de hardware e software, planejamento automatizado e otimização combinatória.

A adoção da forma normal conjuntiva como representação canônica não é arbitrária: ela decorre de propriedades estruturais que facilitam a aplicação de regras de inferência, como a resolução, de modo mecânico e sistemático. Uma fórmula em CNF permite que a verificação de sua satisfazibilidade seja reduzida à manipulação de conjuntos de cláusulas, o que se presta naturalmente à implementação em algoritmos de busca e retrocesso (*backtracking*). Por outro lado, a forma normal disjuntiva, embora menos utilizada em provadores automáticos de teoremas por razões de eficiência computacional, permanece central em áreas como minimização de circuitos digitais, síntese lógica e sistemas de regras em inteligência artificial, nos quais a formulação de uma função como disjunção de termos conjuntivos corresponde diretamente à enumeração de condições suficientes para a verdade de uma proposição.

Cabe ainda destacar que a conversão de uma fórmula arbitrária para forma normal, seja conjuntiva, seja disjuntiva, não é um processo computacionalmente trivial: no pior caso, o tamanho da fórmula resultante pode crescer exponencialmente em relação ao tamanho da fórmula original, o que impõe limites práticos à aplicação ingênua desses procedimentos em sistemas de larga escala. Essa tensão entre a elegância teórica das formas normais e o custo computacional de sua obtenção é um dos fios condutores que atravessam esta dissertação, e será retomada com maior profundidade no Capítulo 2, dedicado aos algoritmos de transformação e à sua análise de correção e complexidade.

1.3 Definições e propriedades: negativa, conjuntiva, disjuntiva e prenexa

Antes de avançar para os algoritmos de conversão propriamente ditos, no Capítulo 2, é necessário fixar com precisão o que se entende por uma *forma normal* e apresentar as quatro formas que estruturam esta dissertação: a forma normal negativa, a forma normal conjuntiva, a forma normal disjuntiva e a forma normal prenexa. Em termos gerais, uma forma normal é uma representação canônica de uma fórmula lógica que obedece a restrições sintáticas específicas quanto à disposição dos conectivos e, no caso da lógica de primeira ordem, dos quantificadores, preservando a equivalência lógica com a fórmula original [12]. Cada uma das quatro formas a seguir impõe um conjunto distinto dessas restrições, e é justamente essa diversidade de critérios que torna necessário tratá-las separadamente antes de discutir, na Seção 1.4, a completude funcional das bases de conectivos que as sustentam.

A **forma normal negativa** (*Negation Normal Form*, NNF) constitui o primeiro passo desse processo de normalização. Uma fórmula está em NNF quando os únicos conectivos presentes são a conjunção, a disjunção e a negação, e esta última incide exclusivamente

sobre fórmulas atômicas, isto é, quando toda subfórmula negada é um literal [12]. A obtenção da NNF a partir de uma fórmula arbitrária passa por dois procedimentos sistemáticos: primeiro, a eliminação dos conectivos de implicação e de bicondicional, reescritos em termos de conjunção, disjunção e negação; segundo, o deslocamento das negações para o interior da fórmula por meio das leis de De Morgan e da eliminação da dupla negação, até que restem apenas negações de literais. Todo formula da lógica proposicional clássica admite uma NNF equivalente, e essa equivalência é o que garante que os passos seguintes de normalização — a saber, a obtenção das formas conjuntiva e disjuntiva — possam partir de uma base sintaticamente controlada, sem alterar o significado da fórmula original.

A partir de uma fórmula em NNF, aplicam-se as leis distributivas da conjunção sobre a disjunção (ou vice-versa) para se obter as formas conjuntiva e disjuntiva. A **forma normal conjuntiva** (*Conjunctive Normal Form*, CNF) é definida como uma conjunção de cláusulas, em que cada cláusula é, por sua vez, uma disjunção de literais: $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_n$, onde cada $C_i = l_1 \vee l_2 \vee \cdots \vee l_k$ e cada l_j é uma variável proposicional ou sua negação. Simetricamente, a **forma normal disjuntiva** (*Disjunctive Normal Form*, DNF) é definida como uma disjunção de termos, em que cada termo é uma conjunção de literais: $\varphi = D_1 \vee D_2 \vee \cdots \vee D_n$, onde cada $D_i = l_1 \wedge l_2 \wedge \cdots \wedge l_k$. Ambas as formas guardam uma relação de dualidade entre si: a negação de uma fórmula em CNF, reescrita em NNF, resulta em uma fórmula em DNF, e vice-versa. Todo formula da lógica proposicional admite uma CNF e uma DNF equivalentes, o que pode ser demonstrado tanto por indução sobre a estrutura da fórmula a partir da sua NNF quanto de forma construtiva a partir de sua tabela-verdade, tomando a disjunção das conjunções correspondentes às linhas em que a fórmula é verdadeira (para a DNF) ou a conjunção das disjunções correspondentes às linhas em que é falsa (para a CNF) [1]. É importante notar, no entanto, que nem a CNF nem a DNF de uma fórmula são únicas: diferentes sequências de aplicação das leis distributivas e comutativas podem produzir formas conjuntivas ou disjuntivas distintas, ainda que logicamente equivalentes entre si, o que tem implicações diretas para a complexidade computacional discutida na Seção 1.2 e retomada no Capítulo 2.

Por fim, a **forma normal prenexa** (*Prenex Normal Form*, PNF) distingue-se das três anteriores por pertencer ao domínio da lógica de primeira ordem, na qual a presença de quantificadores universais e existenciais exige um tratamento sintático adicional. Uma fórmula está em PNF quando assume a forma $Q_1 x_1 \, Q_2 x_2 \cdots Q_n x_n \, M$, em que cada Q_i é um quantificador ($\forall$ ou $\exists$) e M, chamada de matriz da fórmula, não contém quantificadores. A ideia de reunir todos os quantificadores num prefixo antecedendo uma matriz livre de quantificadores remonta ao trabalho de Charles Sanders Peirce, que em 1885, no artigo *On the Algebra of Logic: A Contribution to the Philosophy of Notation*, introduziu regras para deslocar os quantificadores para a esquerda da fórmula, antecipando o que viria a ser

sistematizado como forma normal prenexa [6]. Assim como ocorre com a NNF, a CNF e a DNF, todo fórmula de primeira ordem é logicamente equivalente a uma fórmula em PNF, obtida por meio de regras de equivalência que permitem mover cada quantificador para fora do escopo dos conectivos proposicionais, com o cuidado de renomear variáveis ligadas sempre que necessário para evitar capturas indevidas. Uma vez em PNF, a matriz M pode ainda ser colocada em CNF ou em DNF, resultando nas chamadas formas normais prenexa conjuntiva e prenexa disjuntiva, respectivamente — uma combinação particularmente relevante para os procedimentos de prova automática de teoremas mencionados na Seção 1.2, uma vez que isola, de um lado, a estrutura quantificacional da fórmula e, de outro, sua estrutura proposicional.

Em conjunto, essas quatro formas normais evidenciam um traço comum: todas preservam a equivalência lógica da fórmula original, mas nenhuma delas é única, e a transformação de uma fórmula arbitrária em qualquer uma delas pode acarretar um custo computacional não trivial, como já apontado na Seção 1.2. Essa tensão entre canonicidade sintática, de um lado, e eficiência de obtenção, de outro, é o que motiva a discussão da Seção 1.4, dedicada à completude funcional das bases de conectivos que tornam essas formas possíveis, e será retomada de maneira mais técnica no Capítulo 2, no qual os algoritmos de conversão e sua complexidade computacional são analisados em detalhe.

1.4 Completude funcional: bases de conectivos

As definições e os teoremas de existência apresentados na Seção 1.3 repousam, ainda que implicitamente, sobre uma pressuposição que convém agora explicitar: a de que os conectivos escolhidos — conjunção, disjunção e negação — são suficientes para expressar qualquer função booleana, e portanto qualquer fórmula da lógica proposicional. Essa suficiência não é uma propriedade trivial nem exclusiva desse conjunto particular de conectivos; trata-se de uma propriedade metateórica conhecida como **completude funcional**, e sua investigação é o que permite compreender por que certas bases de conectivos servem de fundamento para as formas normais, enquanto outras não.

Formalmente, um conjunto de conectivos $\mathcal{B}$ é dito funcionalmente completo quando toda função booleana $f : \{0,1\}^n \to \{0,1\}$, para qualquer n, pode ser expressa por uma fórmula construída exclusivamente a partir dos conectivos de $\mathcal{B}$ e de variáveis proposicionais. O conjunto $\{\wedge, \vee, \neg\}$, empregado nas definições de NNF, CNF e DNF na seção anterior, é funcionalmente completo, mas é também redundante: por força das leis de De Morgan, a disjunção pode ser definida a partir da conjunção e da negação (ou vice-versa), de modo que já os conjuntos $\{\wedge, \neg\}$ e $\{\vee, \neg\}$ bastam para expressar qualquer função booleana. A demonstração da completude funcional de conjuntos como esses decorre diretamente do teorema de existência da forma normal disjuntiva discutido na Seção 1.3:

como toda função booleana pode ser lida a partir de sua tabela-verdade e reescrita como uma disjunção das conjunções correspondentes às linhas em que assume valor verdadeiro, qualquer conjunto de conectivos capaz de expressar conjunção, disjunção e negação é, por construção, funcionalmente completo.

O primeiro tratamento sistemático da questão da completude funcional é atribuído a Emil Post que, em sua dissertação de doutoramento de 1920, publicada em 1921 sob o título *Introduction to a General Theory of Elementary Propositions*, não apenas demonstrou a completude do cálculo proposicional de *Principia Mathematica*, de Whitehead e Russell, como também investigou sistematicamente quais subconjuntos de funções booleanas eram fechados sob composição [7]. Esse trabalho deu origem ao que hoje se conhece como reticulado de Post, uma classificação completa de todos os *clones* de funções booleanas — isto é, de todos os conjuntos de funções fechados sob composição e contendo as projeções — da qual decorre, como corolário, um critério necessário e suficiente para a completude funcional de qualquer conjunto de conectivos: um conjunto $\mathcal{B}$ é funcionalmente completo se e somente se não estiver inteiramente contido em nenhuma das cinco classes maximais de funções booleanas identificadas por Post, a saber, as funções que preservam o valor verdadeiro, as que preservam o valor falso, as autoduais, as monótonas e as lineares (afins).

Um resultado particularmente relevante para a computação, e que ilustra de maneira contundente o alcance da noção de completude funcional, é a existência de conectivos isolados que, sozinhos, já constituem uma base funcionalmente completa. O caso mais conhecido é o da *negação conjunta* (NAND), também chamada de traço de Sheffer, introduzida por Henry Sheffer em 1913 como parte de uma axiomatização alternativa da álgebra booleana [9]. A partir do NAND isoladamente é possível definir a negação ($p \uparrow p$), a conjunção ($(p \uparrow q) \uparrow (p \uparrow q)$) e, por extensão, todos os demais conectivos booleanos, o que faz do NAND — e, dualmente, do NOR, por vezes chamado de seta de Peirce — o único conectivo binário necessário para expressar qualquer fórmula proposicional. Essa propriedade não é apenas uma curiosidade lógica: é o fundamento teórico que permite que circuitos digitais inteiros sejam implementados unicamente a partir de portas NAND ou unicamente a partir de portas NOR, uma técnica amplamente empregada em projeto de circuitos integrados, e que retoma diretamente a discussão da Seção 1.2 sobre a aplicação da álgebra booleana aos circuitos de comutação.

A completude funcional, portanto, não é uma questão meramente formal: é ela que garante, de um lado, que as formas normais definidas na Seção 1.3 são capazes de representar qualquer fórmula da lógica proposicional, e, de outro, que a escolha da base de conectivos empregada em uma implementação computacional — seja ela $\{\wedge, \vee, \neg\}$, seja um conectivo único como o NAND — é uma decisão de engenharia e não uma limitação expressiva. Essa distinção entre expressividade teórica e economia de representação

será retomada na Seção 1.5, na qual são discutidas as limitações teóricas que, apesar da completude funcional aqui estabelecida, ainda cercam o uso prático das formas normais.

1.5 Limitações teóricas

As três seções anteriores estabeleceram um panorama aparentemente favorável: toda fórmula proposicional admite uma NNF, uma CNF e uma DNF equivalentes (Seção 1.3), toda fórmula de primeira ordem admite uma PNF equivalente (Seção 1.3), e a completude funcional garante que qualquer base de conectivos suficientemente expressiva — desde $\{\wedge, \vee, \neg\}$ até um único conectivo como o NAND — é capaz de sustentar essa representação (Seção 1.4). Esse panorama, no entanto, diz respeito apenas à existência das formas normais, não ao custo de sua obtenção nem à sua utilidade prática irrestrita. Cabe agora fechar o Capítulo 1 examinando as limitações teóricas que atravessam esse aparato, as quais motivam, em grande medida, os algoritmos de conversão que serão discutidos no Capítulo 2.

A primeira e mais imediata dessas limitações é de natureza combinatória: a conversão ingênua de uma fórmula qualquer para a forma normal conjuntiva ou disjuntiva, por meio da aplicação repetida das leis distributivas sobre uma fórmula em NNF, pode acarretar um crescimento exponencial no número de cláusulas ou de termos em relação ao tamanho da fórmula original. Uma fórmula tão simples quanto $(a_1 \wedge b_1) \vee (a_2 \wedge b_2) \vee \cdots \vee (a_n \wedge b_n)$, já em DNF, produz, quando convertida para CNF por distributividade, uma conjunção com 2^n cláusulas. Esse problema levou Grigori Tseitin a propor, em 1968, um procedimento alternativo de conversão para CNF que introduz uma variável auxiliar para cada subfórmula da expressão original, produzindo uma fórmula cujo tamanho cresce apenas linearmente em relação ao da fórmula de entrada [10]. O preço dessa eficiência, contudo, é uma perda de equivalência estrita: a fórmula obtida pela transformação de Tseitin não é logicamente equivalente à fórmula original, mas apenas *equissatisfazível* a ela — isto é, uma é satisfazível se e somente se a outra também o for —, o que basta para os propósitos de verificação de satisfazibilidade, mas não para todo e qualquer uso das formas normais.

Além do custo de obtenção, há um limite teórico mais profundo relacionado à complexidade computacional dos problemas que essas formas normais foram concebidas para resolver. Stephen Cook demonstrou, em 1971, que o problema de decidir se uma fórmula em CNF é satisfazível — o problema SAT mencionado na Seção 1.2 — é NP-completo, isto é, pertence à classe de problemas mais difíceis dentre aqueles cuja solução pode ser verificada em tempo polinomial [2]. Esse resultado, seminal para a teoria da complexidade computacional, tem uma contrapartida dual e frequentemente menos discutida: decidir se uma fórmula em DNF é uma tautologia — ou seja, se é verdadeira sob toda atribuição de valores — é um problema coNP-completo. A CNF e a DNF, portanto, embora simétricas

do ponto de vista sintático, são radicalmente assimétricas do ponto de vista computacional: cada uma delas é eficiente para decidir um tipo de pergunta (satisfazibilidade, no caso da CNF; validade, no caso da DNF) e, presumivelmente, ineficiente para decidir a pergunta dual, a menos que P = NP.

Uma terceira limitação diz respeito à ausência de um procedimento eficiente para obter a forma normal *mínima* de uma fórmula, isto é, a CNF ou a DNF com o menor número possível de cláusulas ou de termos. Embora a existência de uma forma normal esteja garantida pelos teoremas discutidos na Seção 1.3, encontrar a representação mais compacta entre as múltiplas formas normais equivalentes de uma mesma fórmula — problema historicamente associado à minimização de circuitos lógicos e a métodos como o de Quine–McCluskey — é, em geral, um problema computacionalmente intratável. A não unicidade das formas normais, já apontada na Seção 1.3, não é, portanto, apenas uma curiosidade formal, mas a raiz de um problema de otimização genuinamente difícil.

Por fim, no domínio da lógica de primeira ordem, a forma normal prenexa apresenta uma limitação análoga à observada na transformação de Tseitin: o procedimento de skolemização, frequentemente empregado em conjunto com a PNF para eliminar quantificadores existenciais em provadores automáticos de teoremas, também não preserva a equivalência lógica da fórmula original, apenas sua equissatisfazibilidade, e pode introduzir símbolos funcionais que aumentam a assinatura da linguagem. Além disso, a ordem em que os quantificadores aparecem no prefixo de uma PNF não é arbitrária: alternâncias entre quantificadores universais e existenciais podem impor, a certas classes de problemas, limites inferiores de complexidade que nenhuma reformulação sintática é capaz de contornar.

Em conjunto, essas limitações — o crescimento exponencial da conversão ingênua, a intratabilidade computacional subjacente aos problemas de satisfazibilidade e validade, a dificuldade de minimização e a perda de equivalência estrita nos procedimentos de skolemização — não invalidam a utilidade teórica e prática das formas normais estabelecida ao longo deste capítulo, mas delimitam com precisão o que se pode esperar delas. É justamente esse delicado equilíbrio entre elegância teórica e custo computacional que o Capítulo 2 se propõe a examinar em maior profundidade, ao revisar os principais algoritmos de conversão para formas normais e discutir sua correção, sua complexidade e as decisões de implementação que deles decorrem.

Capítulo 2

Algoritmos de Transformação

2.1 Revisão dos principais métodos de conversão para formas normais

2.2 Correção, equivalência lógica e complexidade computacional

2.3 Considerações sobre implementação computacional

Capítulo 3

Projeto e Implementação da Ferramenta

3.1 Requisitos funcionais e arquitetura da solução

3.2 Módulos de transformação algorítmica

3.3 Interface pedagógica

3.4 Validação: testes automatizados

Capítulo 4

Conclusão

4.1 Síntese dos resultados obtidos

4.2 Contribuições para o ensino de lógica

4.3 Limitações do trabalho realizado

4.4 Perspetivas de evolução e trabalhos futuros

Referências

Bibliografia

[1] Chin-Liang Chang and Richard Char-Tung Lee. *Symbolic Logic and Mechanical Theorem Proving*. Academic Press, New York, 1973.

[2] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC '71)*, pages 151–158, 1971.

[3] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960.

[4] Gilson Pedroso dos Santos e Ronilson dos Santos Bezerra. Desenvolvendo o pensamento computacional utilizando scratch e lógica matemática. `https://conteudos.uniasselvi.com.br/extensao/documentos/2023/artigos/7214_9200_1_SM.pdf`, 2017. Acesso em 23 fev. 2026.

[5] Volker Peckhaus. 19th century logic between philosophy and mathematics. *Bulletin of Symbolic Logic*, 5(4):433–450, 1999.

[6] Charles Sanders Peirce. On the algebra of logic: A contribution to the philosophy of notation. *American Journal of Mathematics*, 7(2):180–202, 1885.

[7] Emil L. Post. Introduction to a general theory of elementary propositions. *American Journal of Mathematics*, 43(3):163–185, 1921.

[8] Claude E. Shannon. A symbolic analysis of relay and switching circuits. *Transactions of the American Institute of Electrical Engineers*, 57(12):713–723, 1938.

[9] Henry M. Sheffer. A set of five independent postulates for Boolean algebras, with application to logical constants. *Transactions of the American Mathematical Society*, 14(4):481–488, 1913.

[10] Grigori S. Tseitin. On the complexity of derivation in propositional calculus. In *Studies in Constructive Mathematics and Mathematical Logic, Part II*, 1968. Reprinted in J. Siekmann and G. Wrightson (eds.), Automation of Reasoning 2, Springer, 1983, pp. 466–483.

[11] Ketrin Diovana Vargas, João Pablo da Silva, and Alice Finger. Estratégias para o ensino de lógica matemática com pensamento computacional: Uma revisão sistemática da literatura. In *Anais do XXXIII Simpósio Brasileiro de Informática na Educação*, pages 1391–1403, Porto Alegre, RS, Brasil, 2022. SBC.

[12] Michal Walicki. *Introduction to Mathematical Logic*. World Scientific, 2011.

[13] Richard Zach. Logic in mathematics and computer science. 2024. Acesso em 22 mar. 2026.

Apêndice A

Scripts de Transformação

A.1 Scripts de Transformação